Módulo extra 2: Interação com o banco de dados

Sumário

Sumário..... 2
1. Conceitos Fundamentais – Mapeando o Mundo para o CRUD e o Node.js.....3
1.1. O que é Persistência de Dados?.....3
1.2. O Pilar de Tudo: O Conceito de CRUD.....3
1.3. Onde o Node.js entra nessa História?..... 4
2. Mão na Massa – Instalando o Node.js e Configurando o Ambiente..... 5
2.1. Instalando o Node.js e o NPM.....5
2.2. Inicializando o Projeto com o NPM (npm init -y)..... 5
2.3. Instalando o Driver de Banco de Dados (npm install mysql2).....6
3. A Ponte para os Dados – Criando a Conexão e Executando o CRUD no Código..... 6
3.1. O Conceito de Connection Pool (Pool de Conexões)..... 7
3.2. Escrevendo o Arquivo de Conexão (database.js)..... 8
3.3. Colocando o CRUD em Prática via Node.js.....8
4. Engenharia Robusta – Boas Práticas na Camada de Dados com Node.js.... 10
4.1. Tratamento de Erros Resiliente e Des Pânico..... 10
4.2. Isolamento de Responsabilidades (Separação de Camadas)..... 12
4.3. Gerenciamento de Conexões e Encerramento Gracioso (Graceful Shutdown) 13
4.4. Uso Estrito de Prepared Statements contra SQL Injection..... 13

1. Conceitos Fundamentais – Mapeando o Mundo para o CRUD e o Node.js

Antes de escrevermos linhas e mais linhas de código de conexão, precisamos entender o propósito conceitual da camada de dados em uma aplicação. No coração de quase todo sistema backend moderno está o conceito de persistência e o acrônimo mais famoso da engenharia de software: o CRUD.

Neste primeiro capítulo, vamos explorar o que significa persistir dados, como o padrão CRUD se traduz em comandos SQL e qual é o papel exato do Node.js nessa engrenagem.

1.1. O que é Persistência de Dados?

As variáveis e objetos que criamos durante a execução de um script em JavaScript vivem exclusivamente na memória RAM. Se a aplicação reiniciar, se houver uma queda de energia ou se o servidor for reinicializado, tudo desaparece.

A persistência é a capacidade de salvar esses dados em um meio de armazenamento duradouro (um disco rígido, SSD ou um servidor de banco de dados na nuvem), garantindo que eles continuem existindo e acessíveis ao longo do tempo.

É aqui que entram os SGBDs (Sistemas de Gerenciamento de Bancos de Dados), como o MySQL. Eles organizam os dados de forma estruturada (em tabelas, colunas e linhas) e garantem que múltiplos processos possam ler e gravar informações simultaneamente com segurança.

1.2. O Pilar de Tudo: O Conceito de CRUD

Independentemente do tamanho da aplicação (seja uma rede social, um sistema bancário ou um painel de controle industrial), todas as interações com um banco de dados se resumem a quatro operações fundamentais, conhecidas pela sigla CRUD:

- C - Create (Criar / Inserir): Adicionar novos registros de forma permanente na base de dados.
    - Exemplo conceitual: Cadastrar um novo usuário, registrar um pedido de compra ou salvar um log de erro.
    - Comando SQL equivalente: INSERT INTO
- R - Read (Ler / Consultar): Buscar e recuperar os dados armazenados para exibi-los ou utilizá-los nas regras de negócio.
    - Exemplo conceitual: Consultar o perfil de um cliente, listar o estoque de produtos ou buscar o histórico de transações.

    - Comando SQL equivalente: SELECT

- U - Update (Atualizar / Modificar): Alterar informações de registros que já existem no banco de dados.

    - Exemplo conceitual: Mudar o endereço de e-mail de um usuário, atualizar o status de entrega de um pacote ou redefinir uma senha.

    - Comando SQL equivalente: UPDATE

- D - Delete (Excluir / Remover): Apagar permanentemente um registro da base de dados.

    - Exemplo conceitual: Cancelar uma conta de usuário, remover um item do carrinho de compras ou limpar registros antigos de auditoria.

    - Comando SQL equivalente: DELETE

1.3. Onde o Node.js entra nessa História?

O banco de dados sabe armazenar e organizar o CRUD perfeitamente, mas ele não sabe conversar com a interface do usuário (como o navegador ou um aplicativo mobile) e nem possui a lógica de negócio da sua aplicação.

O Node.js atua como a ponte lógica e o motor de controle:

- Tradução de Intenções: Quando o usuário clica em "Salvar Cadastro" na interface, o Node.js recebe essa requisição, valida se os dados estão corretos, formata a operação e a envia para o banco de dados.

- Segurança e Regras: É o Node.js quem decide se a operação pode acontecer (verificando permissões, sanitizando dados e blindando o sistema contra ataques como SQL Injection).

- Tratamento de Respostas: O Node.js recebe os dados brutos devolvidos pelo banco de dados (por exemplo, as linhas de um SELECT) e os transforma em estruturas limpas para serem consumidas.

2. Mão na Massa – Instalando o Node.js e Configurando o Ambiente

No Capítulo 1, entendemos o propósito conceitual da persistência, o funcionamento do CRUD e por que o Node.js atua como a ponte inteligente entre a aplicação e o banco de dados. Agora, é hora de trazer esses conceitos para a prática.

Neste capítulo, vamos realizar a instalação do Node.js, criar o nosso diretório de trabalho do zero e configurar as ferramentas essenciais para interagir com o banco de dados.

2.1. Instalando o Node.js e o NPM

O primeiro passo é garantir que o interpretador JavaScript e o gerenciador de pacotes estejam disponíveis no seu sistema operacional.

- Baixando o instalador: Acesse o site oficial do Node.js (nodejs.org) e baixe a versão LTS (Long Term Support), que é a mais estável e recomendada para ambientes de desenvolvimento e produção.

- Executando a instalação: Siga o assistente de instalação padrão (Next, Next, Finish), aceitando as configurações sugeridas. O instalador configurará automaticamente as variáveis de ambiente necessárias.

- Validando a instalação: Abra o seu terminal (Prompt de Comando, PowerShell ou terminal Linux/macOS) e digite os seguintes comandos para verificar se tudo ocorreu bem:

```
node -v
npm -v
```

Se o terminal retornar os números das versões instaladas (por exemplo, v20.x.x para o Node e 10.x.x para o NPM), significa que o seu ambiente está pronto.

2.2. Inicializando o Projeto com o NPM (npm init -y)

Com o Node.js instalado, precisamos criar uma pasta dedicada para o nosso projeto de banco de dados e dar uma "identidade" a ela.

- Crie uma pasta no seu computador para o projeto (por exemplo, projeto-banco-node) e abra essa pasta pelo terminal.

- Execute o comando de inicialização do gerenciador de pacotes:

```
npm init -y
```

O que aconteceu aqui?

O comando gerou um arquivo chamado package.json na raiz da sua pasta.

Por que isso é importante? O package.json é o documento de identidade do seu projeto. Ele registra o nome da aplicação, a versão e, principalmente, a lista de dependências (as bibliotecas externas que o seu código vai precisar para funcionar). A flag -y apenas automatizou a criação, aceitando as configurações padrão.

2.3. Instalando o Driver de Banco de Dados (npm install mysql2)

O JavaScript puro não sabe se comunicar nativamente com os protocolos de rede de um banco de dados relacional como o MySQL. Para fazer essa ponte, precisamos instalar um driver otimizado.

No terminal, dentro da pasta do projeto, execute:

```
npm install mysql2
```

O que aconteceu aqui?

O NPM baixou a biblioteca mysql2 (uma das mais rápidas e eficientes para Node.js, com suporte nativo a Promises e async/await).

Ele criou uma pasta oculta chamada node_modules (onde os arquivos da biblioteca ficam salvos) e registrou o mysql2 automaticamente dentro das dependências do seu arquivo package.json.

3. A Ponte para os Dados – Criando a Conexão e Executando o CRUD no Código

No Capítulo 2, preparamos o nosso ambiente: instalamos o Node.js, inicializamos o projeto com o package.json e equipamos a aplicação com o driver mysql2.

Agora que a ferramenta está pronta, chegou o momento de tirar a teoria do papel e construir a ponte que liga o Node.js ao banco de dados relacional, implementando as operações do CRUD na prática.

3.1. O Conceito de Connection Pool (Pool de Conexões)

Antes de escrevermos código, precisamos responder a uma pergunta fundamental de engenharia de software: Como o Node.js deve se conectar ao banco de dados?

Uma abordagem ingênua seria abrir uma nova conexão com o banco toda vez que uma função precisasse consultar um dado, fechando-a logo em seguida. Em sistemas com múltiplos acessos simultâneos, isso gera um gargalo severo de desempenho, consumindo muita CPU, memória e tempo de rede.

A solução profissional é utilizar um Connection Pool (Pool de Conexões):

- O pool abre um grupo de conexões reutilizáveis logo na inicialização da aplicação e as mantém em descanso.

- Quando o Node.js precisa executar um comando SQL, ele pega uma conexão emprestada do pool, executa a operação em frações de segundo e a devolve imediatamente para o grupo, pronta para o próximo uso.

3.2. Escrevendo o Arquivo de Conexão (database.js)

Crie um arquivo chamado database.js na raiz do seu projeto. É aqui que centralizaremos toda a configuração de acesso ao SGBD.

```
// database.js
const mysql = require('mysql2/promise');

// Criando o pool de conexões com o banco de dados
const pool = mysql.createPool({
    host: 'localhost',       // Endereço onde o banco está rodando
    user: 'root',            // Usuário do banco de dados
    password: 'sua_senha',   // Senha de acesso
    database: 'seu_banco',   // Nome do banco de dados
    waitForConnections: true,
    connectionLimit: 10,     // Número máximo de conexões
simultâneas no pool
    queueLimit: 0
});

module.exports = pool;
```

Ao exportar o pool, qualquer outro arquivo do nosso projeto poderá requisitá-lo para executar comandos SQL de forma limpa e organizada.

3.3. Colocando o CRUD em Prática via Node.js

Agora vamos criar um arquivo principal (por exemplo, app.js) para testar as quatro operações fundamentais do CRUD. Como o acesso a bancos de dados é assíncrono (leva alguns milésimos de segundo para ir e voltar pela rede), utilizaremos a estrutura async/await.

A. Criando Registros (C - Create com INSERT)

Lembre-se da regra de ouro do backend: nunca concatene strings diretamente nas queries para evitar falhas de segurança como o SQL Injection. Utilizamos os Prepared Statements com interrogações (?) como marcadores de parâmetros.

```
// app.js
const db = require('./database');

async function cadastrarUsuario(nome, email) {
    try {
        const sql = 'INSERT INTO usuarios (nome, email) VALUES (?,
?)';
```

```
        // O segundo argumento é um array com os valores que
substituem as interrogações
        const [resultado] = await db.execute(sql, [nome, email]);

        console.log(`Usuário cadastrado com sucesso! ID gerado:
${resultado.insertId}`);
    } catch (error) {
        console.error('Erro ao cadastrar usuário:',
error.message);
    }
}
```

B. Lendo Registros (R - Read com SELECT)

Para buscar os dados armazenados, executamos uma consulta e capturamos as linhas retornadas pelo banco.

```
async function listarUsuarios() {
    try {
        const sql = 'SELECT id, nome, email FROM usuarios';

        // O primeiro elemento do array retornado pelo execute são
as linhas da tabela
        const [linhas] = await db.execute(sql);

        console.log('Lista de Usuários no Banco:');
        console.table(linhas); // Exibe os dados formatados em
tabela no terminal
    } catch (error) {
        console.error('Erro ao listar usuários:', error.message);
    }
}
```

C. Atualizando Registros (U - Update com UPDATE)

Modificamos informações existentes aplicando sempre a cláusula WHERE para garantir que apenas o registro correto seja alterado.

```
async function atualizarEmailUsuario(id, novoEmail) {
    try {
        const sql = 'UPDATE usuarios SET email = ? WHERE id = ?';

        const [resultado] = await db.execute(sql, [novoEmail,
```

```
id]);

        if (resultado.affectedRows > 0) {
            console.log(`E-mail atualizado com sucesso para o
usuário ID ${id}!`);
        } else {
            console.log(`Nenhum usuário encontrado com o ID
${id}.`);
        }
    } catch (error) {
        console.error('Erro ao atualizar:', error.message);
    }
}
```

D. Excluindo Registros (D - Delete com DELETE)

Removemos um registro indesejado com base em seu identificador único.

```
async function deletarUsuario(id) {
    try {
        const sql = 'DELETE FROM usuarios WHERE id = ?';

        const [resultado] = await db.execute(sql, [id]);

        console.log(`Usuário ID ${id} removido com sucesso.`);
    } catch (error) {
        console.error('Erro ao deletar:', error.message);
    }
}
```

4. Engenharia Robusta – Boas Práticas na Camada de Dados com Node.js

Nos capítulos anteriores, instalamos o Node.js, configuramos o pool de conexões e implementamos o CRUD básico. No entanto, escrever código que "funciona" é apenas o primeiro passo. Em ambientes de produção reais, a camada de dados precisa ser resiliente, limpa e segura contra falhas inesperadas.

Neste capítulo, vamos mergulhar nas boas práticas essenciais de engenharia de software para interagir com bancos de dados usando Node.js.

4.1. Tratamento de Erros Resiliente e Des Pânico

Em uma aplicação conectada a um banco de dados, muitas coisas podem dar errado: a rede pode cair, o servidor de banco de dados pode reiniciar, uma chave

única pode ser duplicada ou uma tabela pode não existir. Se você não tratar os erros adequadamente, sua aplicação Node.js vai quebrar (crash).

A Má Prática:

```
// NUNCA faça isso: deixar o erro sem tratamento ou apenas dar um
console.log simples
async function buscarUsuarioRuim(id) {
    const [linhas] = await db.execute('SELECT * FROM usuarios
WHERE id = ?', [id]);
    return linhas[0]; // Se a conexão cair, a aplicação morre aqui
}
```

A Boa Prática:

Utilize blocos try/catch estruturados, diferencie erros operacionais (como dados inválidos) de erros de infraestrutura, e forneça mensagens claras sem expor detalhes sensíveis do banco para o usuário final.

```
async function buscarUsuarioSeguro(id) {
    try {
        const [linhas] = await db.execute('SELECT id, nome, email
FROM usuarios WHERE id = ?', [id]);

        if (linhas.length === 0) {
            console.warn(`Aviso: Usuário com ID ${id} não foi
encontrado.`);
            return null;
        }

        return linhas[0];
    } catch (error) {
        // Log detalhado para o desenvolvedor no servidor
        console.error(` Erro crítico ao buscar usuário ID ${id}:`,
error.message);

        // Lança um erro customizado ou retorna um padrão seguro
para a camada superior
        throw new Error('Não foi possível processar a consulta no
momento.');
    }
}
```

4.2. Isolamento de Responsabilidades (Separação de Camadas)

Misturar lógica de banco de dados diretamente com rotas HTTP ou scripts de automação deixa o código confuso, difícil de testar e impossível de manter. A boa prática dita que devemos isolar as operações de banco de dados em módulos dedicados (muitas vezes chamados de Repositories ou Services).

Estrutura de Arquivos Limpa:

```
meu-projeto/
│
├── database.js          # Apenas a configuração do pool de
conexões
├── userRepository.js    # Apenas queries SQL referentes à tabela
de usuários
└── app.js               # Apenas a execução ou chamada das
regras
```

Exemplo do userRepository.js:

```
const db = require('./database');

const UserRepository = {
    async criar(nome, email) {
        const sql = 'INSERT INTO usuarios (nome, email) VALUES (?,
?)';
        const [resultado] = await db.execute(sql, [nome, email]);
        return resultado.insertId;
    },

    async listarTodos() {
        const [linhas] = await db.execute('SELECT id, nome, email
FROM usuarios');
        return linhas;
    }
};

module.exports = UserRepository;
```

Dessa forma, se amanhã você decidir mudar de MySQL para PostgreSQL ou trocar o driver, você altera o código em um único arquivo, sem quebrar o resto da aplicação.

4.3. Gerenciamento de Conexões e Encerramento Gracioso (Graceful Shutdown)

Quando o seu servidor Node.js recebe um comando para desligar (por exemplo, durante um deploy ou uma atualização de infraestrutura), ele não deve simplesmente cortar a energia abruptamente. Se houver consultas em andamento ou conexões abertas no pool, elas podem corromper dados.

A boa prática exige implementar o encerramento gracioso do pool de conexões:

```
// server.js ou app.js
const db = require('./database');

// Captura o sinal de encerramento do sistema operacional (ex:
Ctrl+C ou sinal de kill do Docker/Kubernetes)
process.on('SIGINT', async () => {
    console.log('\nEncerrando a aplicação de forma graciosa...');
    try {
        // Fecha todas as conexões pendentes do pool de forma
segura
        await db.end();
        console.log('Pool de conexões fechado com sucesso.');
        process.exit(0);
    } catch (error) {
        console.error('Erro ao fechar o pool de conexões:',
error.message);
        process.exit(1);
    }
});
```

4.4. Uso Estrito de Prepared Statements contra SQL Injection

Vale a pena reforçar esta que é a regra de ouro número um de segurança em bancos de dados: Jamais faça interpolação de strings em queries SQL.

Errado e Perigoso: `const query = \SELECT * FROM usuarios WHERE email = '${userInput}'`;`

Certo e Blindado: `const [linhas] = await db.execute('SELECT * FROM usuarios WHERE email = ?', [userInput]);`

O uso de parâmetros isolados (?) garante que o motor do banco trate a entrada do usuário estritamente como dado, e nunca como parte executável do código SQL, eliminando por completo o risco de SQL Injection.